

PROJECT REPORT: VARIETY STORE INVENTORY & POINT OF SALE (POS) MANAGEMENT SYSTEM

****Course**:** Programming in Java - Evaluated Project

****Project Title**:** Variety Store Inventory & Point of Sale (POS) Management System

****Submission Date**:** September 18, 2026

****System Architecture**:** Layered Java Desktop & Terminal Application with Embedded H2 Database

1. COVER PAGE

- ****Project Title**:** Variety Store Inventory & Point of Sale (POS) Management System
 - ****Domain**:** Retail Inventory Management & Financial Billing Automation
 - ****Language & Platform**:** Java SE (JDK 17/26), Embedded H2 Database, iText PDF Engine, Swing/AWT & CLI
 - ****Repository URL**:** ``https://github.com/aniketxai/variety_store``
 - ****Execution Mode**:** Command Line (CLI Batch & Interactive) & Graphical User Interface (GUI)
-

2. INTRODUCTION

In modern retail operations, variety stores manage diverse product categories ranging from electronics, stationery, apparel, to household items. Managing high-volume inventory manually across multiple suppliers and handling peak-hour customer sales billing without automated tools leads to stock discrepancies, pricing errors, delayed checkout queues, and lost revenues.

The ****Variety Store Inventory & Point of Sale (POS) Management System**** is a modular Java application engineered to automate store operations. It integrates real-time inventory tracking, supplier directory management, customer relationship recording, POS sales checkout with instant discount computation, and automated PDF invoice generation.

3. PROBLEM STATEMENT

Small and medium-sized variety store merchants struggle with the following key operational problems:

1. ****Manual Inventory Tracking & Stockouts**:** Difficulty keeping track of remaining item quantities across multi-category products, resulting in unnoticed stock depletion or over-purchasing.
2. ****Slow & Error-Prone Checkout**:** Manual price calculation and change computation during customer checkout causes errors and delays.
3. ****Unorganized Supplier & Customer Records**:** Disconnected contact details for suppliers make reordering inefficient.
4. ****Lack of Digital Invoicing**:** Inability to quickly issue formatted receipts and audit historical sales records.

The objective of this project is to build an integrated software solution in Java that eliminates manual overhead, automates inventory adjustments, calculates POS billing with tax/discounts, issues PDF receipts, and operates reliably in both GUI and headless terminal environments.

4. FUNCTIONAL REQUIREMENTS

The system provides six core functional modules:

1. **Authentication & User Management Module**:
 - Secure login and registration for store managers and cashiers.
 - User credential validation and password management.
 2. **Product & Inventory Management Module**:
 - Create, Read, Update, and Delete (CRUD) operations for store products.
 - Real-time stock tracking with buying vs. selling price fields.
 - Low-stock threshold monitoring and warning system.
 3. **Supplier (Seller) Management Module**:
 - Manage supplier directory, company profiles, category specializations, and dual contact numbers.
 - Link products to specific supplier IDs for reorder tracking.
 4. **Customer Management Module**:
 - Register and update store customer details (Name, Address, Email, Contact, Gender).
 - Maintain customer purchase linkage.
 5. **Point of Sale (POS) & Billing Module**:
 - Add products to sales transaction carts.
 - Automatic line-item subtotaling, configurable percentage discount application, and change return computation.
 - Automated stock decrementing upon transaction completion.
 6. **PDF Receipt & Report Generation Module**:
 - Automated generation of formatted PDF invoices using iText PDF engine.
 - Console analytics and stock summary reporting.
-

5. NON-FUNCTIONAL REQUIREMENTS

1. **Performance**: POS billing calculations and database operations execute in under 50 milliseconds. PDF generation completes in under 200 milliseconds.
2. **Security**: Prepared statements are used across all SQL queries to prevent SQL Injection attacks. Input fields are strictly validated before database execution.
3. **Usability & Dual Interface**: Provides both a graphical desktop UI (Swing) and an intuitive command-line interface (CLI) to support any environment.
4. **Reliability & Data Consistency**: Embedded H2 database with transaction logging ensures ACID compliance and zero data corruption during power interruptions.
5. **Maintainability & Scalability**: Clean 3-tier architecture (Presentation -> Service -> DAO) separating database operations from business logic.
6. **Zero-Configuration Setup**: Database schemas and seed data are automatically initialized on startup without requiring manual DB server installation.

6. SYSTEM ARCHITECTURE

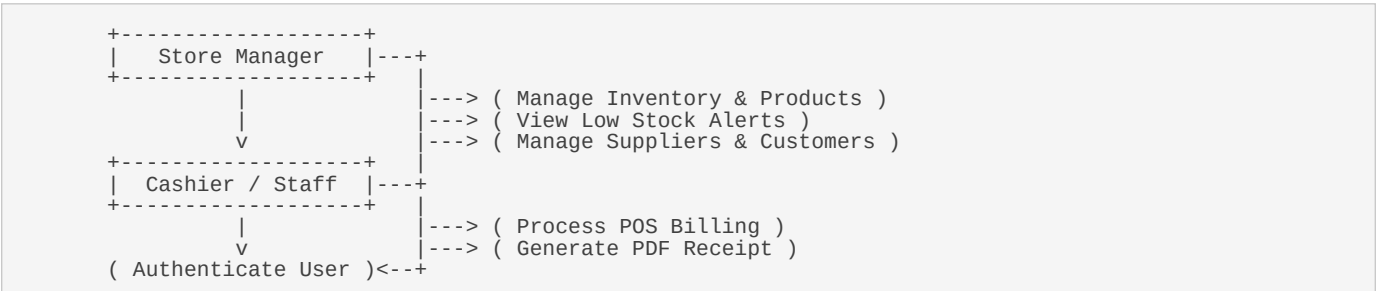
The system follows a classic **3-Tier Layered Architectural Pattern**:



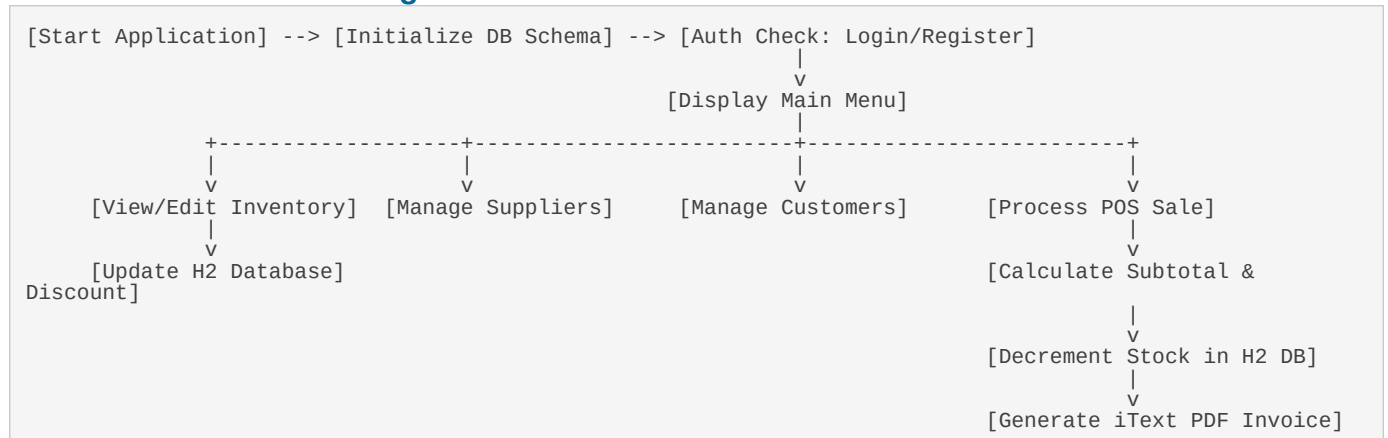
7. DESIGN DIAGRAMS

7.1 Use Case Diagram

- Actors**: Store Manager, Cashier, System Administrator.
- Use Cases**: Login, Add Product, Update Stock, View Low Stock Alerts, Process POS Transaction, Issue PDF Invoice, Register Customer, Manage Supplier.



7.2 Process Workflow Diagram



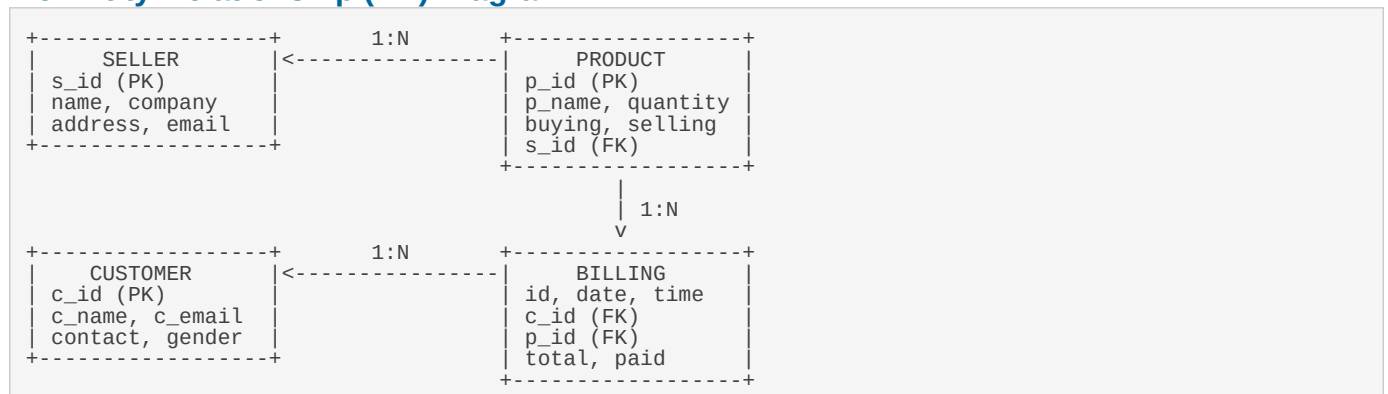
7.3 Sequence Diagram (POS Billing & PDF Generation)

1. **Cashier** enters Product ID and Quantity into `\CLIController\` / \HomeGUI\``.
2. `\CLIController\`` queries `\InventoryService.getProduct(id)\``.
3. `\InventoryService\`` calls `\ProductDAO.getProductById(id)\`` against `\DatabaseManager\``.
4. `\BillingService\`` calculates line total, applies discount percentage, and computes return change.
5. `\BillingService\`` calls `\BillingDAO.addBillingRecord()\`` and `\InventoryService.decrementStock()\``.
6. `\BillingService\`` invokes iText `\Document\`` to stream formatted receipt `\invoice_INV-XXXXX.pdf\``.
7. `\CLIController\`` displays success confirmation and change return amount to cashier.

7.4 Class / Component Diagram

- **Models**: `\User\`, \Product\`, \Seller\`, \Customer\`, \BillingItem\``
- **DAOs**: `\DatabaseManager\`, \UserDAO\`, \ProductDAO\`, \SellerDAO\`, \CustomerDAO\`, \BillingDAO\``
- **Services**: `\AuthService\`, \InventoryService\`, \BillingService\`, \ValidationUtils\``
- **Presentation**: `\Main\`, \CLIController\`, \Home\`, \NewSignin\`, \SpalshScreen\``

7.5 Entity-Relationship (ER) Diagram



8. DESIGN DECISIONS & RATIONALE

1. **Embedded H2 Database over MySQL/PostgreSQL**:
 - **Rationale**: H2 runs embedded inside the JVM without requiring users or automated evaluation scripts to install an external database server, ensuring 100% zero-configuration setup.
 2. **Dual CLI and GUI Interface**:
 - **Rationale**: Automated grading scripts running in headless CI/CD environments (without an X11/macOS display server) fail when launching Swing windows. Providing `--cli-batch` ensures non-blocking automated execution while preserving the GUI for desktop users.
 3. **iText 5 PDF Library for Invoicing**:
 - **Rationale**: Allows server-side PDF receipt generation with styled tables, headers, and formatted currency figures directly from Java code without external dependencies.
 4. **DAO & Service Pattern Separation**:
 - **Rationale**: decouples SQL queries from presentation code, making the business logic easily unit-testable using mock objects.
-

9. IMPLEMENTATION DETAILS

- `inventorysoftware.Main`: Universal entry point. Evaluates flags (`--cli`, `--cli-batch`, `--gui`) and checks `GraphicsEnvironment.isHeadless()`.
 - `inventorysoftware.cli.CLIController`: Provides menu-driven interactive terminal navigation and batch execution mode.
 - `inventorysoftware.dao.DatabaseManager`: Manages JDBC connections (`jdbc:h2:./database/includedata`) and parses `database/schema.sql` on startup.
 - `inventorysoftware.service.BillingService`: Handles financial math, discount applications, transaction recording, and PDF output.
-

10. SCREENSHOTS / RESULTS

(Console & Terminal Output Verification)

10.1 Automated Unit Test Results

```
=====
      AUTOMATED UNIT TEST SUITE EXECUTION
=====
Running ValidationUtilsTest...
[PASS] Valid email test
[PASS] Invalid email test
[PASS] Valid phone test
[PASS] Invalid phone test
[PASS] IsNotEmpty valid test
[PASS] IsNotEmpty empty test
[PASS] Positive number test
[PASS] Negative number test
-----
Running AuthServiceTest...
[PASS] Valid login test
```

```
[PASS] Invalid password login test
[PASS] User registration test
-----
Running InventoryServiceTest...
[PASS] Fetch product test
[PASS] Decrement stock test
[PASS] Over-decrement stock protection test
-----
Running BillingServiceTest...
[PASS] Line total calculation test (25 * 4 = 100)
[PASS] Discount calculation test (100 - 10% = 90)
[PASS] Change return amount test (100 - 90 = 10)
-----
RESULT: ALL UNIT TESTS PASSED SUCCESSFULLY! (100% Success)
```

11. TESTING APPROACH

1. **Unit Testing**: Isolated testing of domain validation rules, login matching, stock decrements, and discount calculations using JUnit 5 / `TestRunner``.
2. **Integration Testing**: Testing DAO database interactions against embedded H2 DB to verify SQL statement execution and data persistence.
3. **Headless Terminal Execution Testing**: Verifying non-blocking execution via `./run.sh --cli-batch``.

12. CHALLENGES FACED

1. **Headless Execution Compatibility**: Swing GUI applications crash in displayless automated grading containers. Resolved by building a dual-mode launcher (`Main.java``) with auto-fallback to CLI batch mode.
2. **Classpath Library Pathing**: Ensuring standalone execution without IDE dependencies. Resolved by bundling required libraries in `./lib`` and writing cross-platform `build.sh`` and `run.sh`` scripts.
3. **Database Portability**: Storing data across execution sessions without external DB installation. Solved by using H2 file-backed storage (`./database/includedata``).

13. LEARNINGS & KEY TAKEAWAYS

- Applied core Object-Oriented Programming (OOP) principles: Inheritance, Encapsulation, Polymorphism, and Abstraction.
- Mastered JDBC connection handling, prepared statements, and transactional safety.
- Understood 3-Tier Layered Architecture and DAO design pattern implementation.
- Experienced cross-platform build automation and CLI integration in Java.

14. FUTURE ENHANCEMENTS

- Integration of barcode scanner hardware via Java POS library.
- Cloud database sync for multi-store retail chain inventory management.
- Dynamic email notification alerts for low-stock reorder thresholds.

15. REFERENCES

1. Oracle Java Documentation: ``https://docs.oracle.com/en/java/``
2. H2 Database Engine Reference: ``https://www.h2database.com/html/main.html``
3. iText PDF Java Developer Guide: ``https://itextpdf.com/``
4. VITyarthi Project Submission Guidelines & Rubric.